

section 3.5: Loops -- While and For

page 60

Remember that, as always, the *statement* can be a brace-enclosed block.

Make sure you understand how the `for` loop

```
for (expr<sub>1</sub>; expr<sub>2</sub>; expr<sub>3</sub>)
    statement
```

is equivalent to the `while` loop

```
expr<sub>1</sub>;
while (expr<sub>2</sub>) {
    statement
    expr<sub>3</sub>;
}
```

There is nothing magical about the three expressions at the top of a `for` loop; they can be arbitrary expressions, and they're evaluated just as the expansion into the equivalent `while` loop would suggest. (Actually, there are two tiny differences: the behavior of `continue`, which we'll get to in a bit, and the fact that the test expression, `expr<sub>2</sub>`, is optional and defaults to ```true"` for a `for` loop, but is required for a `while` loop.)

`for(;;)` is one way of writing an infinite loop in C; the other common one is `while(1)`. Don't worry about what a `break` would mean in a loop, we'll be seeing it in a few more pages.

pages 60-61

Deep sentences:

Whether to use `while` or `for` is largely a matter of personal preference...
Nonetheless, it is bad style to force unrelated computations into the initialization and increment of a `for`, which are better reserved for loop control operations.

In general, the three expressions in a `for` loop should all manipulate (initialize, test, and increment) the same variable or data structure. If they don't, they are ```unrelated computations,"` and a `while` loop would probably be clearer. (The reason that one loop or the other can be clearer is simply that, when you see a `for` loop, you *expect* to see an idiomatic initialize/test/increment of a single variable or data structure, and if the `for` loop you're looking at doesn't end up matching that pattern, you've been momentarily misled.)

page 61

When the authors say that ```the index and limit of a C for loop can be altered from within the loop,"` they mean that a loop like

```
int i, n = 10;
for(i = 0; i < n; i++) {
    if(i == 5)
        i++;
    printf("%d\n", i);
    if(i == 8)
        n++;
}
```

```
}
```

where `i` and `n` are modified within the loop, is legal. (Obviously, such a loop can be very confusing, so you'll probably be better off not making use of this freedom too much.)

When they say that ``the index variable... retains its value when the loop terminates for any reason," you may not find this too surprising, unless you've used other languages where it's *not* the case. The fact that loop control variables retain their values after a loop can make some code much easier to write; for example, the `atoi` function at the bottom of this page depends on having its `i` counter manipulated by several loops as it steps over three different parts of the string (whitespace, sign, digits) with `i`'s value preserved between each step.

Deep sentence:

Each step does its part, and leaves things in a clean state for the next.

This is an extremely important observation on how to write clean code. As you study the `atoi` code, notice that it falls into three parts, each implementing one step of the pseudocode description: *skip white space*, *get sign*, *get integer part and convert it*. At each step, `i` points at the next character which that step is to inspect. (If a step is skipped, because there is no leading whitespace or no sign, the later steps don't care.)

You may hear the term *invariant* used: this refers to some condition which exists at all stages of a program or function. In this case, the invariant is that `i` always points to the next character to be inspected. Having some well-chosen invariants can make code *much* easier to write and maintain. If there aren't enough invariants--if `i` is sometimes the next character to look at and sometimes the character that was just looked at--debugging and maintaining the code can be a nightmare.

In the `atoi` example, the lines

```
for (i = 0; isspace(s[i]); i++) /* skip white space */
    ;
```

are about at the brink of ``forcing unrelated computations into the initialization and increment," especially since so much has been forced into the loop header that there's nothing left in the body. It would be equally clear to write this part as

```
i = 0;
while (isspace(s[i]))
    i++;                                /* skip white space */
```

The line

```
sign = (s[i] == '-') ? -1 : 1;
```

may seem a bit cryptic at first, though it's a textbook example of the use of `?:`. The line is equivalent to

```
sign = 1;
if(s[i] == '-')
    sign = -1;
```

pages 61-62

It's instructive to study this Shell or ``gap" sort, but don't worry if you find it a bit bewildering.

Deep sentence:

Notice how the generality of `for` makes the outer loop fit the same form as the others, even though it is not an arithmetic progression.

The point is that loops don't have to count 0, 1, 2... or 1, 2, 3... . (This one counts $n/2$, $n/4$, $n/8$ Later we'll see loops which don't step over numbers at all.)

page 63

Deep sentence:

The commas that separate function arguments, variables in declarations, etc. are *not* comma operators...

This looks strange, but it's true. If you say

```
for (i = 0, j = strlen(s)-1; i < j; i++, j--)
```

the first comma says to do `i = 0` *then* do `j = strlen(s)-1`, and the second comma says to do `i++` *then* do `j--`. However, when you say

```
getline(line, MAXLINE);
```

the comma just separates the two arguments `line` and `MAXLINE`; they both have to be evaluated, but it doesn't matter in which order, and they're both passed to `getline`. (If the comma in a function call were interpreted as a comma operator, the function would only receive one argument, since the value of the first operand of the comma operator is discarded.) Since the comma operator discards the value of its first operand, its first operand had better have a side effect. The expression

```
++a, ++b
```

increments `a` and increments `b` and (if anyone cares) returns `b`'s value, but the expression

```
a+1, b+1
```

adds 1 to `a`, discards it, and returns `b+1`.

If the comma operator isn't making perfect sense, don't worry about it for now. You're most likely to see it in the first or third expression of a `for` statement, where it has the obvious meaning of separating two (or more) things to do during the initialization or increment step. Just be careful that you don't accidentally write things like

```
for(i = 0; j = 0; i < n && j < j; i++; j++)    /* WRONG */
```

or

```
for(i = 0, j = 0, i < n && j < j, i++, j++)    /* WRONG */
```

The correct form of a multi-index loop is something like

```
for(i = 0, j = 0; i < n && j < j; i++, j++)
```

Semicolons always separate the initialization, test, and increment parts; commas may appear *within*

the initialization and increment parts.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail](#) [feedback](#)